SummarAIze — Text Summarization using Flask & AWS

Cloud Architecture Design

N SRI AHILESH
23BCE0961
G1+TG1

Abstract

The SummarAIze project is a cloud-based text summarization application developed using Flask and AWS. It allows users to upload text files or paste content to generate concise summaries automatically. The project integrates AWS EC2 for deployment and AWS S3 for cloud storage, ensuring scalability and reliability.

Introduction

Text summarization is the process of condensing large pieces of text into shorter versions while retaining the key ideas. This project leverages cloud technologies to provide an accessible and efficient text summarization service. The objective is to automate the summarization process using a pre-trained deep learning model deployed on AWS infrastructure.

Architecture / Workflow

1. The Flask web application allows users to upload text files or paste text directly.
2. The uploaded file is temporarily saved on the EC2 instance.
3. Using AWS S3 integration, the file is uploaded to a specific S3 bucket for storage.
4. The text is then passed to a summarization model for processing.
5. The summarized output is displayed to the user and can be downloaded.
6. Gunicorn serves as the WSGI server, while Nginx acts as a reverse proxy for production deployment.

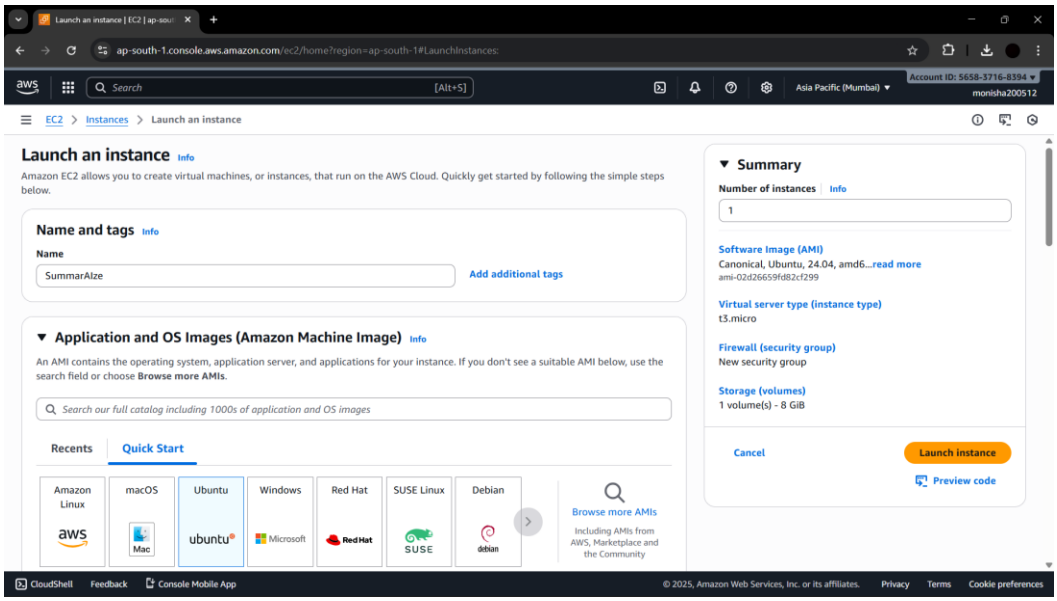
Implementation

The Flask backend handles text input, summarization, and file management. Boto3 is used to connect and upload files to the AWS S3 bucket. Gunicorn is configured as a system service to run the Flask app continuously, and Nginx forwards incoming requests to Gunicorn. Environment variables such as S3 bucket name and AWS credentials are securely stored and used within the application.

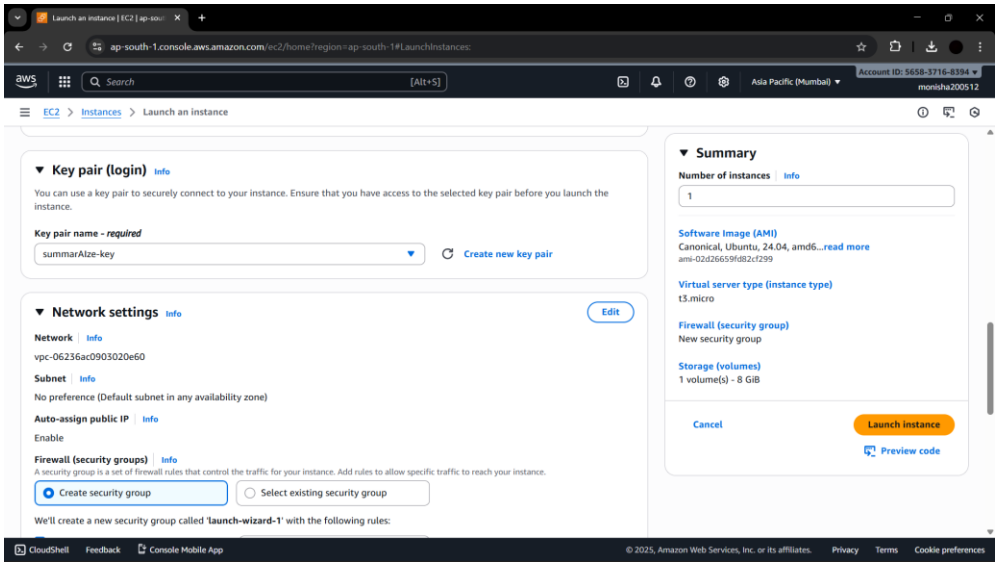
AWS Services Used

- EC2: Hosts the Flask backend in a secure and scalable environment.
- S3: Stores uploaded files and generated summaries.
- IAM: Provides controlled access permissions between EC2 and S3.

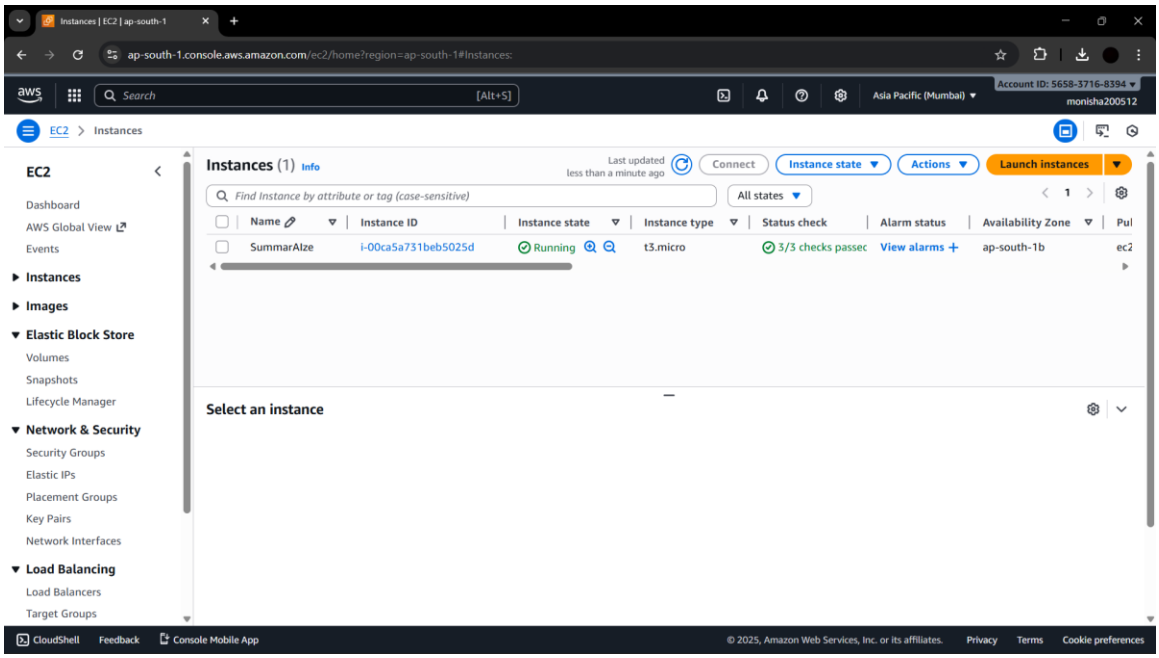
1) Create a new Instance



2) Create a new key-Pair and add security Groups and click launch instance.



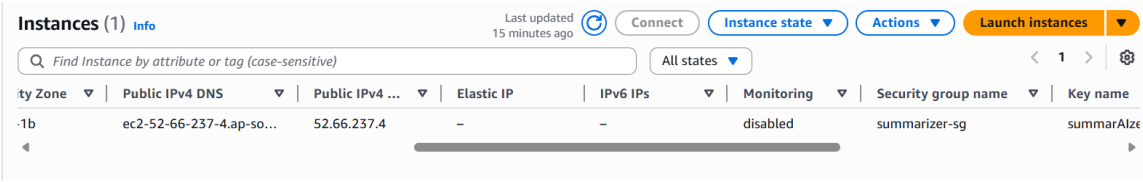
3) Make sure it Passes all status

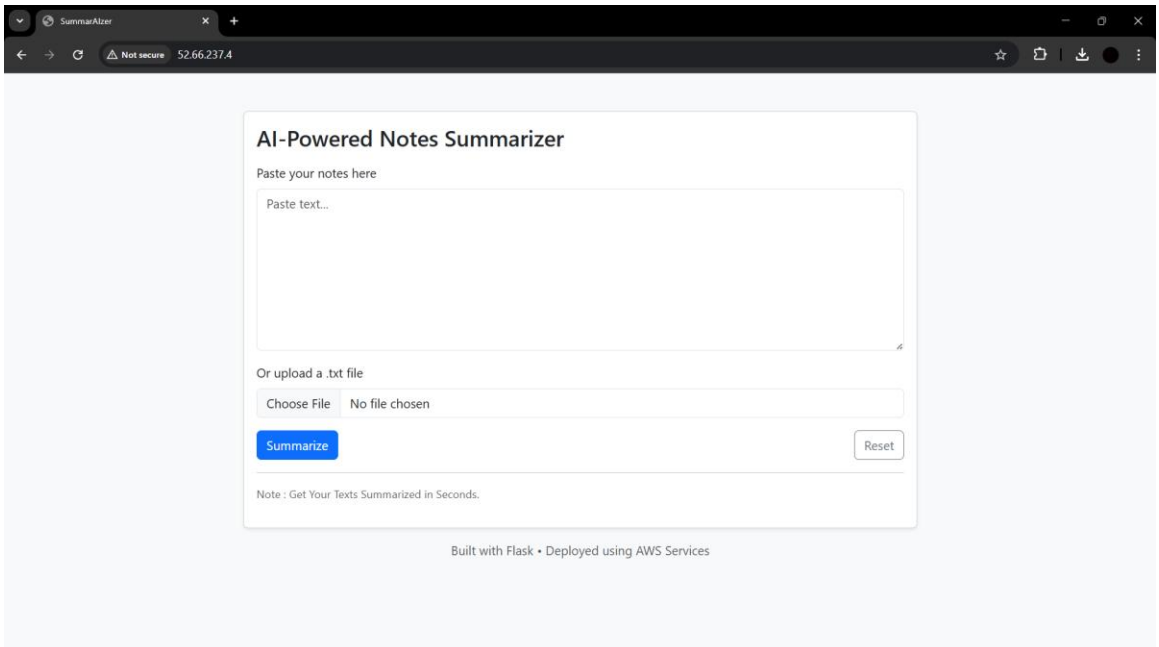


4) Connect the Ec2 instance with the Key pair in your terminal and download all the dependencies , upload all the Project Files inside the Ec2 instance .

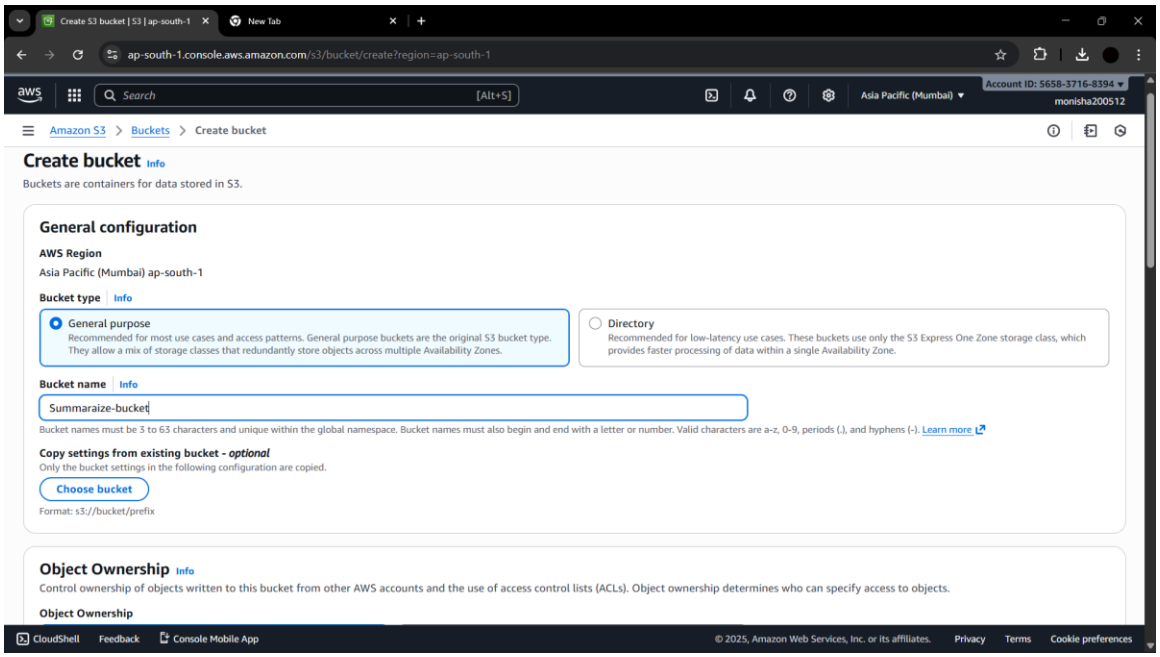
ssh -i summarAIze-key.pem ubuntu@<52.66.237.4>

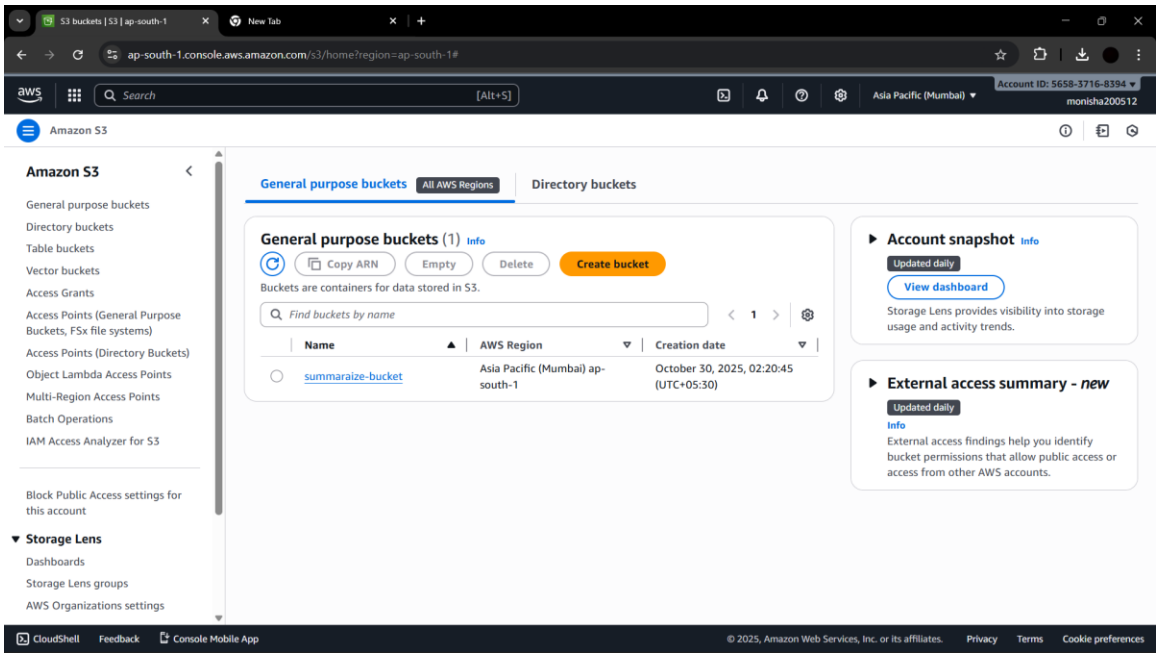
Once if the Website starts running on the Ip of Ec2 instance . It is very well setup





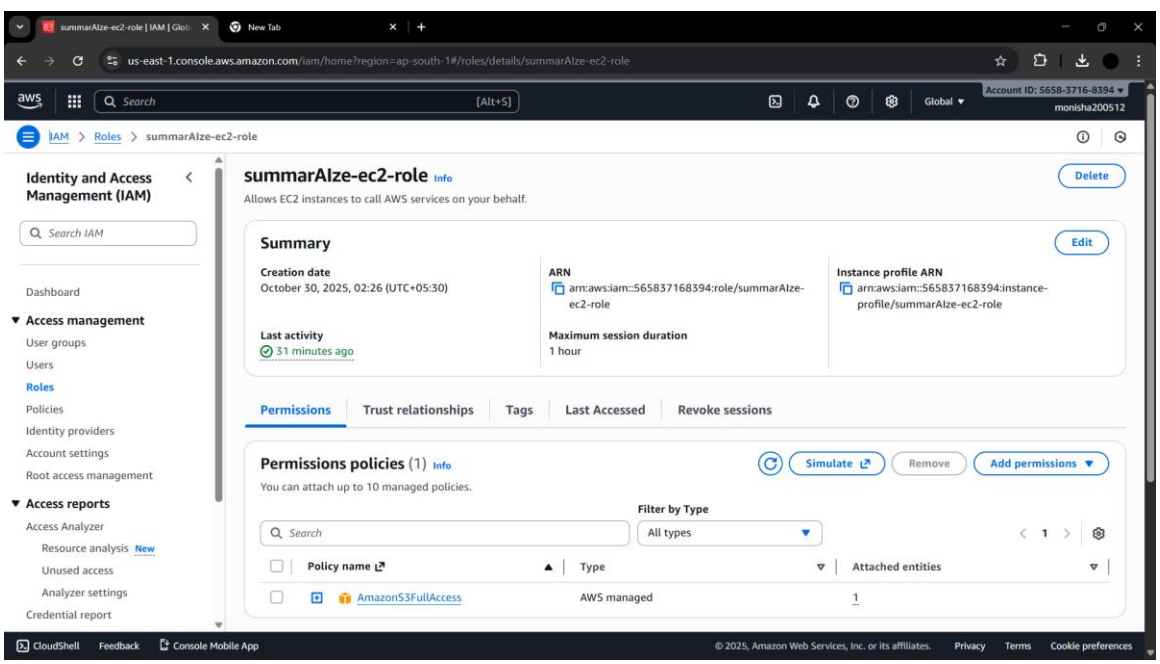
5) Now Create a S3 Bucket , to Save the uploaded Files automatically when uploaded in the Website .



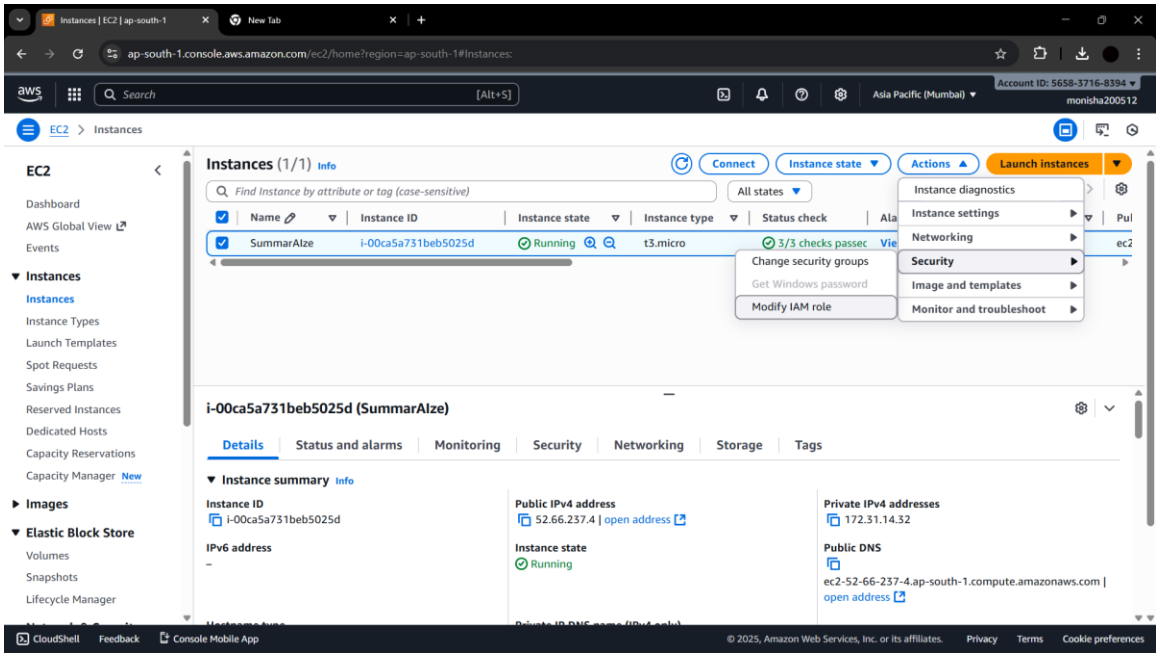


6) Now Create an IAM role to Give Full access of S3 to the Ec2 Instance.

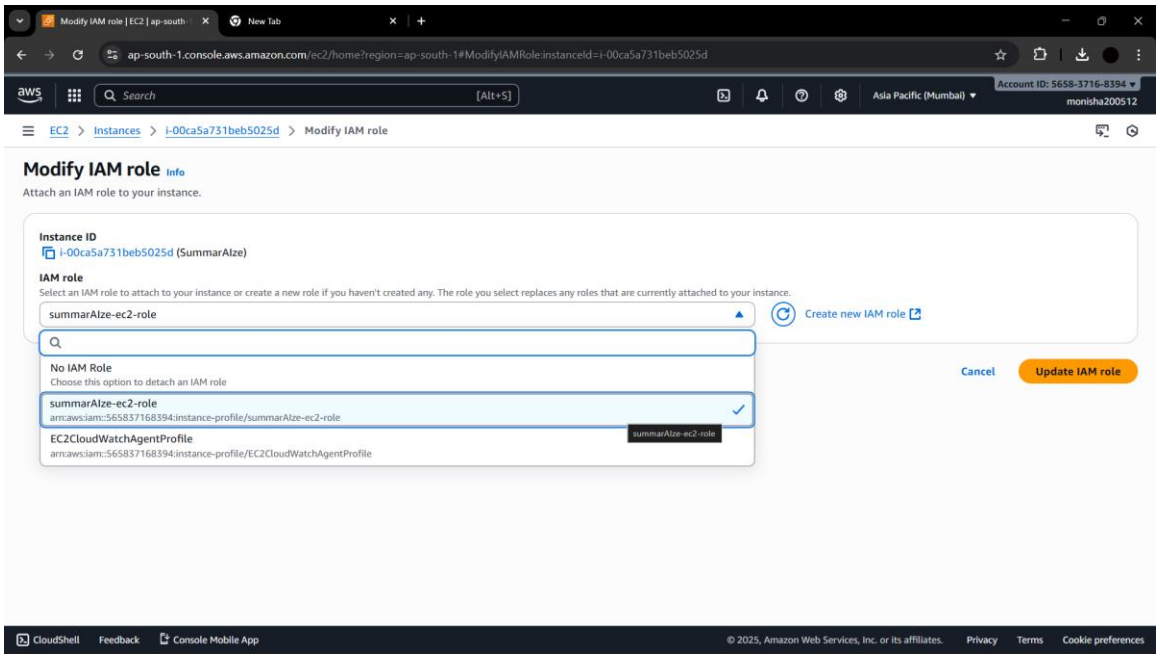
Create a new Role , and add policy to AmazonS3FullAccesss



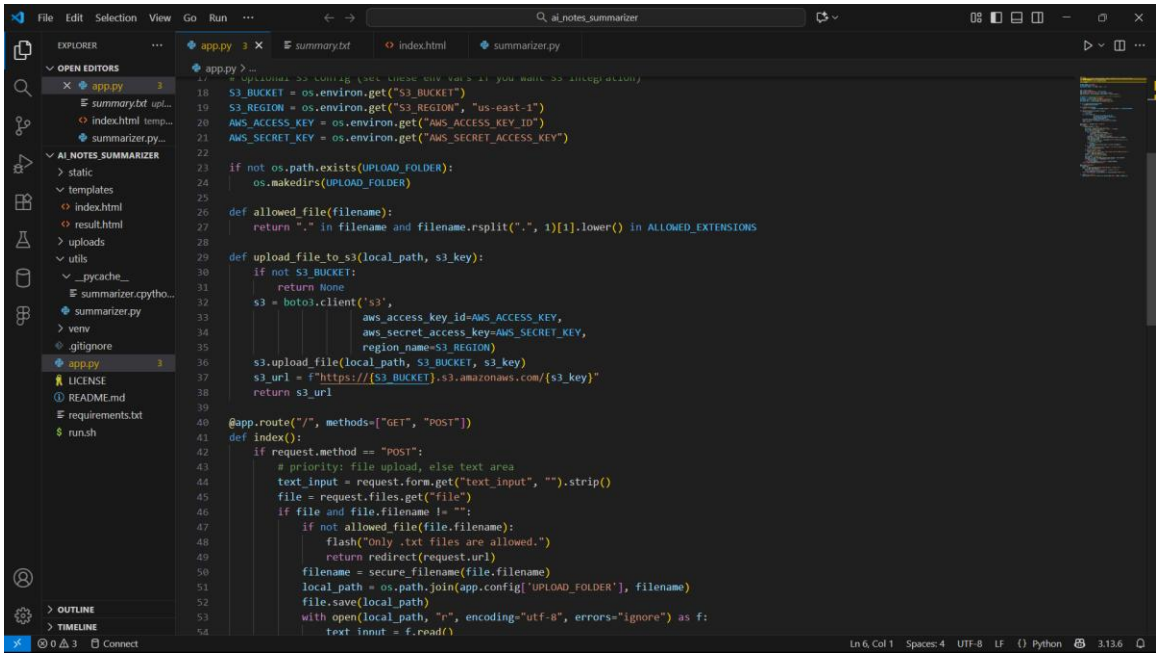
7) Now Open your Ec2 instance and Select Modify IAM Role and choose the newly created IAM role for the Ec2 instance to give full access of the S3 Bucket.



Choose the Created IAM Role and update

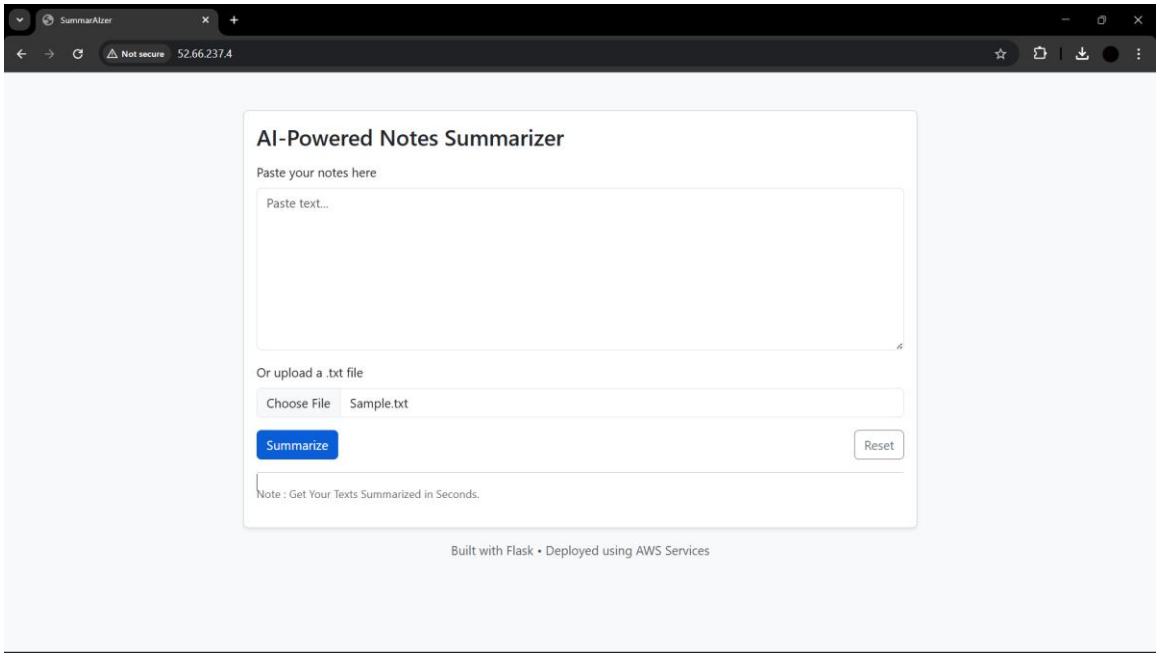


8) Now Add the Code in your Script to automatically save the uploaded files into the S3 Bucket.

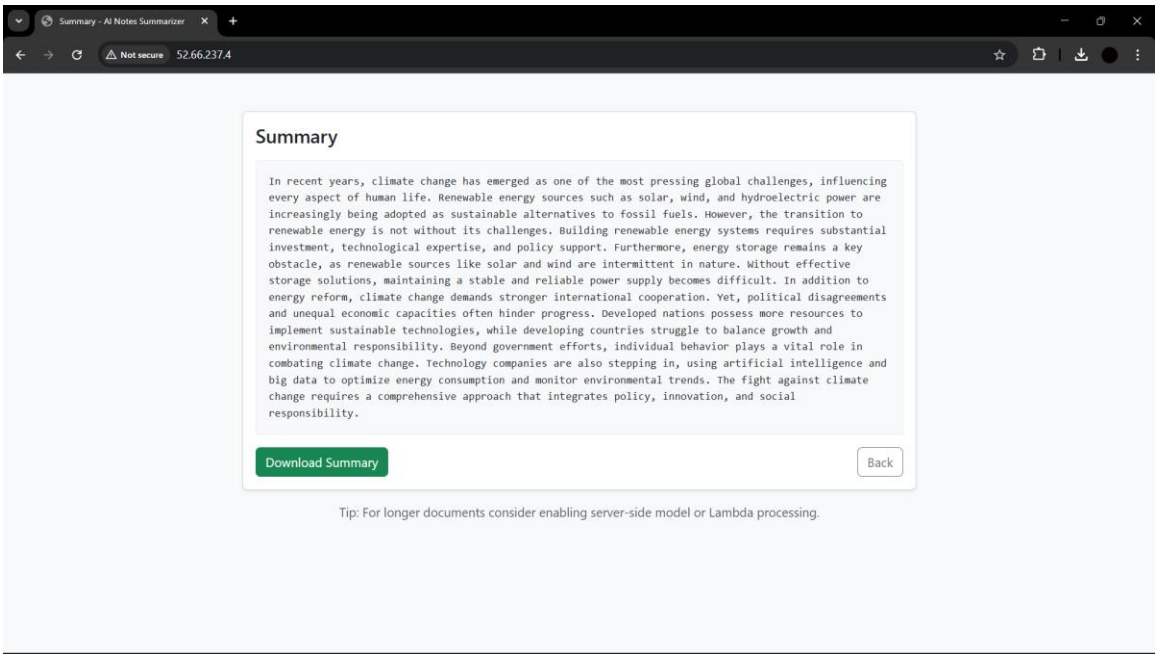



```python
def upload_file_to_s3(local_path, s3_key):
    if not S3_BUCKET:
        return None
    s3 = boto3.client('s3',
                      aws_access_key_id=AWS_ACCESS_KEY,
                      aws_secret_access_key=AWS_SECRET_KEY,
                      region_name=S3_REGION)
    s3.upload_file(local_path, S3_BUCKET, s3_key)
    s3_url = f"https://{S3_BUCKET}.s3.amazonaws.com/{s3_key}"
    return s3_url
```

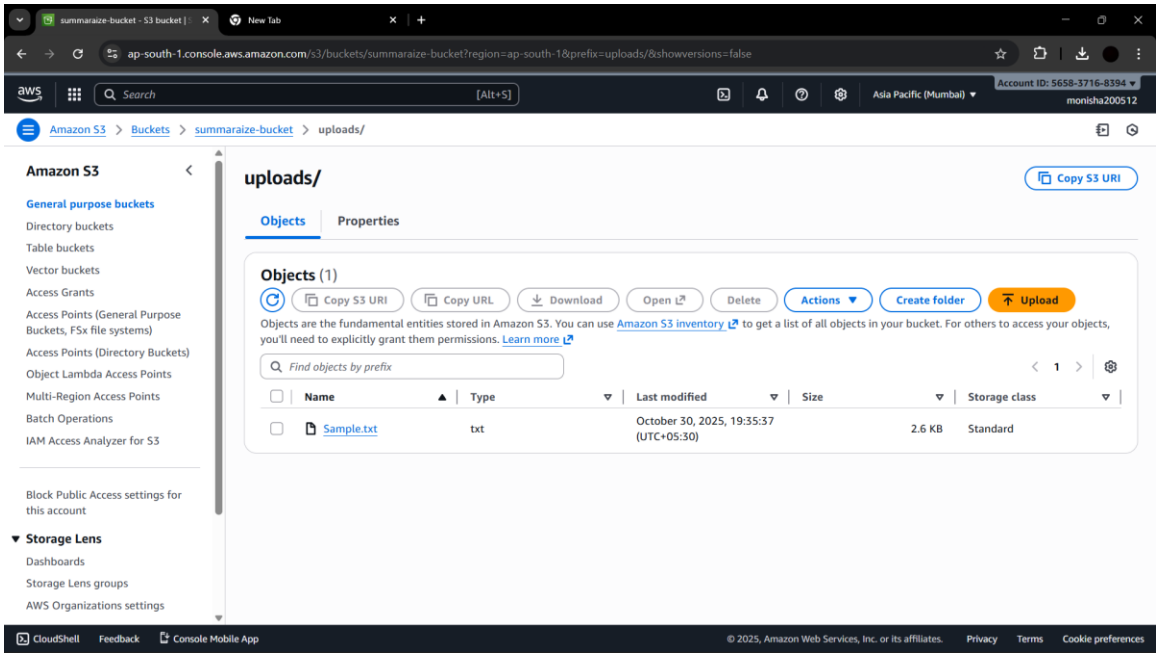
9) All Set. Now if the File is Uploaded for Summarization , It automatically gets saved in the S3 Bucket .



10) Summarized Text.



11) The Sample.txt File is stored inside the S3 Bucket Automatically. Even when the Instance is Stopped, The Files inside S3 are safer and good to go.



Results & Output

The Website is hosted on Ip of the Ec2 instance by using gunicorn and Nginx .

Upon uploading a .txt file, the system generates a summarized version of the input text. Files are successfully stored in the S3 bucket under the 'uploads/' folder. The summarized text is displayed on the webpage and can be downloaded as a file.

Conclusion

The SummarAIze project demonstrates the integration of artificial intelligence and cloud technologies. It automates the process of summarizing text documents using a pre-trained NLP model hosted on AWS. By leveraging services like EC2, S3, and IAM, the system ensures scalability, security, and reliability.